



Data Structures And Algorithms

Name : Samreen Zia
Submitted to : Mr. Rehan Ahmed
Semester : BSE (3A)
Date : 15- Dec- 2025

LAB NO 10

Code task 1:

Implement the (class) Circular Linked List to create a list of integers. You need to provide the implementation of the member functions as described in the following. Use DOUBLY LINKED LIST

class CList

{

private:

struct node

{int data;

node *next;

node *pre;

} *head;

public:

1CList();

// Checks if the list is empty or not

2bool emptyList();

// Inserts a new node with value 'value' after position 'pos'in the list

6void insert (int pos, int value);

// Inserts a new node at the start of the list

3void insert_begin(int value);

// Inserts a new node at the end of the list

5void insert_end(int value);

// Deletes a node from the beginning of the list

7void delete_begin();

// Deletes a node from the end of the list

```

8void delete_end();
    // Deletes a node having specific value
9void delete (int);
    // Displays the values stored in the list
4void traverse();
    // Return the max values stored in the list
10int max();
    // Delete complete list except first and last node
11void two_node_list();
    // Displays the values stored in the list in reverse order
12void traverse2();
};

```

Program:

```

#include <iostream>
using namespace std;

class CList
{
private:
    struct node
    {
        int data;
        node *next;
        node *pre;
    } *head;

public:
    CList()
    {
        head = NULL;
    }
    bool emptyList()
    {
        return (head == NULL);
    }
    void insert_begin(int value)
    {
        node *newNode = new node;
        newNode->data = value;

        if (emptyList())
        {
            newNode->next = newNode;
            newNode->pre = newNode;

```

```

        head = newNode;
    }
    else
    {
        node *last = head->pre;
        newNode->next = head;
        newNode->pre = last;
        last->next = newNode;
        head->pre = newNode;
        head = newNode;
    }
}

```

```

void traverse()
{
    if (emptyList())
    {
        cout << "List is empty\n";
        return;
    }
    node *temp = head;
    do
    {
        cout << temp->data << " ";
        temp = temp->next;
    } while (temp != head);

    cout << endl;
}

```

```

void insert_end(int value)
{
    if (emptyList())
    {
        insert_begin(value);
        return;
    }
    node *newNode = new node;
    newNode->data = value;

    node *last = head->pre;

    newNode->next = head;
    newNode->pre = last;
}

```

```

        last->next = newNode;
        head->pre = newNode;
    }
void insert(int pos, int value)
{
    if (emptyList())
    {
        cout << "List is empty\n";
        return;
    }

    node *temp = head;
    int count = 1;

    while (count < pos)
    {
        temp = temp->next;
        if (temp == head)
        {
            cout << "Invalid position\n";
            return;
        }
        count++;
    }

    node *newNode = new node;
    newNode->data = value;

    newNode->next = temp->next;
    newNode->pre = temp;

    temp->next->pre = newNode;
    temp->next = newNode;
}
void delete_begin()
{
    if (emptyList())
    {
        cout << "List is empty\n";
        return;
    }
    if (head->next == head)
    {

```

```

        delete head;
        head = NULL;
    }
    else
    {
        node *last = head->pre;
        node *temp = head;

        head = head->next;
        head->pre = last;
        last->next = head;

        delete temp;
    }
}
void delete_end()
{
    if (emptyList())
    {
        cout << "List is empty\n";
        return;
    }
    if (head->next == head)
    {
        delete head;
        head = NULL;
    }
    else
    {
        node *last = head->pre;
        node *secondLast = last->pre;

        secondLast->next = head;
        head->pre = secondLast;

        delete last;
    }
}
void delete_val(int value)
{
    if (emptyList())
    {
        cout << "List is empty\n";
        return;
    }

```

```

    }
    node *temp = head;
    do
    {
        if (temp->data == value)
        {
            if (temp == head)
                delete_begin();
            else
            {
                temp->pre->next = temp->next;
                temp->next->pre = temp->pre;
                delete temp;
            }
            return;
        }
        temp = temp->next;
    } while (temp != head);

    cout << "Value not found\n";
}

int max()
{
    if (emptyList())
        return -1;

    int maxVal = head->data;
    node *temp = head->next;

    while (temp != head)
    {
        if (temp->data > maxVal)
            maxVal = temp->data;
        temp = temp->next;
    }
    return maxVal;
}

void two_node_list()
{
    if (emptyList() || head->next == head)
        return;

    node *first = head;
    node *last = head->pre;

```

```

    node *temp = first->next;
    while (temp != last)
    {
        node *del = temp;
        temp = temp->next;
        delete del;
    }
    first->next = last;
    last->pre = first;

    last->next = first;
    first->pre = last;
}
void traverse2()
{
    if (emptyList())
    {
        cout << "List is empty\n";
        return;
    }
    node *temp = head->pre;
    do
    {
        cout << temp->data << " ";
        temp = temp->pre;
    } while (temp != head->pre);

    cout << endl;
}
};

```

```

int main()
{
    CList cl;

    cl.insert_begin(10);
    cl.insert_end(20);
    cl.insert_end(30);
    cl.insert_begin(5);

    cout << "List (forward): ";
    cl.traverse();
}

```

```

cout << "List (reverse): ";
cl.traverse2();

cl.insert(2, 15); // insert 15 after position 2
cout << "After inserting 15 at position 2: ";
cl.traverse();

cl.delete_begin();
cout << "After deleting from beginning: ";
cl.traverse();

cl.delete_end();
cout << "After deleting from end: ";
cl.traverse();

cl.delete_val(15);
cout << "After deleting value 15: ";
cl.traverse();

cout << "Maximum value in list: " << cl.max() << endl;

cl.two_node_list();
cout << "After deleting all except first & last: ";
cl.traverse();

return 0;
}

```

OUTPUT:

```

List (forward): 5 10 15 20 30
List (reverse): 30 20 15 10 5
After inserting 15 at position 2: 5 10 15 15 20 30
After deleting from beginning: 10 15 15 20 30
After deleting from end: 10 15 15 20
After deleting value 15: 10 15 20
Maximum value in list: 20
After deleting all except first & last: 10 20

```